FACULTY OF ENGINEERING AND BASIC SCIENCES
ACADEMIC PROGRAM: DATA ENGINEERING AND ARTIFICIAL INTELLIGENCE

COURSE: ETL (G51)
ETL-Project: Final Delivery

Deadline

The project must be submitted through the platform by the date discussed in class.

Teammate

You must collaborate with your colleagues, which means you must form a group of at least 2 people and no more than 5.

Objectives

This final delivery extends the pipeline developed in the second delivery. You must preserve and build upon the existing project, integrating the three data sources, the Airflow-orchestrated ETL process, data quality validation, and the dimensional model. You will then train and evaluate a machine learning model and integrate its inference into a Kafka-based streaming flow. The project must include two complementary forms of data consumption: a static analytical report generated with Python and a real-time dashboard displaying model predictions.
The goal is to demonstrate how one data architecture can support analytical decision-making and an operational prediction workflow. Reuse and improve the work from the previous delivery rather than creating an unrelated project. The ML model must be appropriate for the group's data and use case; it must not be an unrelated model added only to satisfy the requirement.

What is Expected

1. Data sources

- Continue with the project developed for the second delivery. Do not replace it with a new or unrelated project.

- Use at least three data sources established in the second delivery: (1) the dataset or source used in the first delivery, (2) the additional dataset introduced in the second delivery, and (3) the selected public API.
- The Airflow pipeline must extract or access all three sources. If a source is already stored in a database, the pipeline may query that database rather than re-ingesting the original file.
- Explain how the three sources contribute to the project objectives and how they are linked or combined.
- If a source is no longer available, document the issue and provide a justified, reproducible alternative that preserves the original project logic.

2. Airflow DAG

- Implement or update an Airflow DAG with clear tasks for extraction, transformation, integration/merge, data quality validation, and loading.
- The pipeline must process data from all three sources and integrate them into an enriched dataset.
- Keep the pipeline reproducible and document task dependencies, scheduling, and relevant configuration.
- Reuse the existing Airflow and data-quality work from the second delivery, improving it where necessary. Do not omit validation simply because the project now includes Kafka.
- You must include the respective EDA notebooks created in previous deliveries.

3. Data quality and dimensional model

- Retain Great Expectations or the equivalent data-quality tool used in the second delivery. Define checks that are meaningful for the project and execute them as part of the Airflow DAG.
- Record validation results. The pipeline should prevent invalid data from being loaded into the final analytical tables when critical checks fail.
- Design and load a documented dimensional model in a database, such as a star schema with fact and dimension tables.
- The fact table must contain the metric or measure that will be published to Kafka. Document the metric, its meaning, its grain, and the relevant fields included in each event.
- Ensure that the model is built from transformed and validated data. Explain how duplicate loads or repeated DAG runs are handled.

4. Kafka producer and streaming

- Implement a Python Kafka producer that publishes records containing the input features required by the trained model. The messages should be derived from the project's data and represent a meaningful use case.
- For this assignment, you do not need to implement complex Change Data Capture (CDC) or a production-grade event platform. You may simulate the stream by reading fact-table records and publishing them to a Kafka topic with a short delay between messages.
- Each event should have a documented structure and include the fields required for inference, plus an event identifier or timestamp where appropriate. Do not include the target value as an input feature if doing so would leak the answer.
- Document the Kafka topic, message format, and how the producer obtains the data from the fact table.

5. Machine learning model inference

- Define a prediction problem that is relevant to the project and supported by the available data. Identify the target variable and explain why predicting it is useful.
- Prepare the training dataset using appropriate feature selection and preprocessing. Document how missing values, categorical variables, and other required transformations are handled.
- Train at least one suitable machine learning model. Use a defensible train/test split or another appropriate evaluation strategy, and avoid data leakage.
- Serialize the trained model and any required preprocessing steps so that the Kafka consumer can load and use the same fitted pipeline for inference. Do not retrain the model for every incoming message.
- Document the model, target variable, features, training process, and inference limitations in the README or technical report.

6. Static analytical report

- Create a static analytical report using Python. It must query the validated dimensional model in the database.
- The report is a point-in-time analytical output, not a real-time dashboard. It does not need to refresh automatically or display incoming Kafka messages.

- The report should answer questions related to the project's objectives and present insights, not merely a collection of charts.

7. Kafka consumer and real-time prediction dashboard

- Develop a separate Python application that consumes messages from the Kafka topic, loads the serialized model and preprocessing pipeline, and generates a prediction for each valid incoming message.
- Connect the consumer to a Python-friendly visualization framework, such as Streamlit, Dash, or Bokeh, and display the predictions as messages are processed.
- The dashboard must update as messages are consumed.
- The real-time dashboard must be driven by the Kafka consumer, not by repeatedly querying the dimensional database and labeling the result as a Kafka stream.

8. Deliverables

- Provide Include a brief technical document that explains:
  - Updated architecture diagram,
  - Description of the ETL pipeline steps,
  - Airflow DAG design.
  - Visualizations and insights.
  - Value Generation Articulation: An explicit explanation of how your architecture generates value in two ways: how it supports an operational process (via the real-time Kafka stream) and an analytical process (via the dimensional model and static dashboard).
  - Summary of data transformations, quality checks, and the dimensional model.
  - Kafka topic and message structure, producer/consumer design, and how inference is performed for each incoming message.
  - Static report: questions addressed, visualizations, findings, model-performance analysis where appropriate, and instructions to regenerate it.
  - Model: prediction objective, target and features, preprocessing, serialized artifacts, and limitations.
  -
- Upload your code and diagrams to GitHub as a portfolio project.
  - README.md explaining your project, setup instructions, and key decisions.

- Visualizations
- .gitignore file.

Technology Stack

- Python
- Jupyter Notebook
- Database (you choose)
- Airflow
- Kafka
- Great expectations or equivalent
- Streamlit, Dash, Bokeh, or equivalent
- CSV files and API consumption
- Python visualization or reporting libraries
- Git

Diagram

Example diagram is provided in figure 1 (overall project flow).

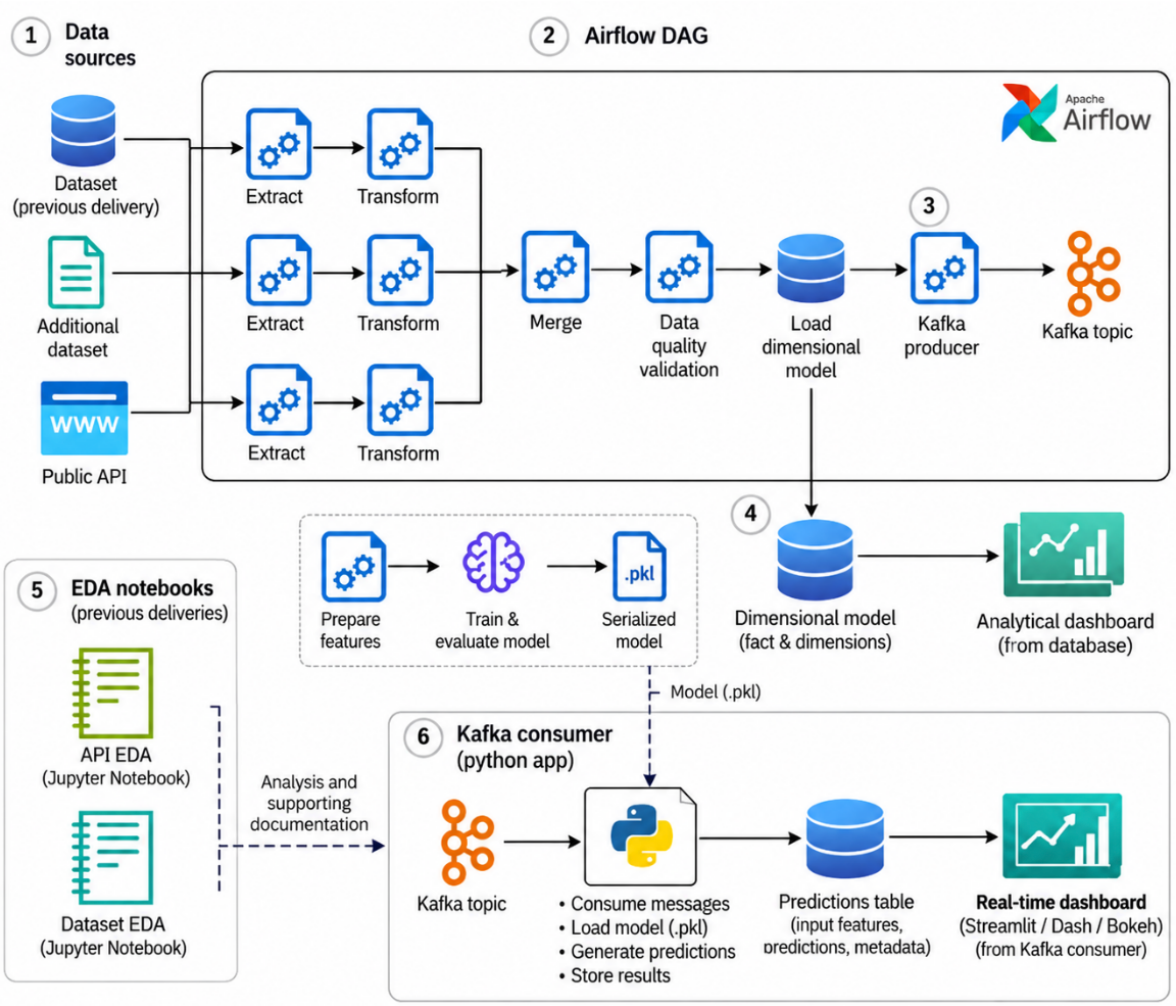


Figure 1. Block Diagram

Criteria evaluation

Item	Description	Weight
Data extraction	Successfully accesses and processes all three sources established in the second delivery. The source roles and extraction approach are documented.	8%
Orchestration	Implements a fully functional Airflow DAG with distinct tasks configured to extract, transform, and load the data. Integrating Great Expectations or an equivalent tool, recording results, and prevents critical validation failures from reaching final analytical tables	10%
Transformation and dimensional model	Integrates data from the three sources and loads a documented dimensional model, including a fact table and a well-defined prediction dataset.	7%
Machine learning model	Defines a relevant target and features, trains a suitable model, and documents limitations	15%
Kafka producer	Publishes valid feature records required for inference to Kafka. The message structure and stream simulation are documented and reproducible.	10%
Kafka consumer	Loads the serialized model and preprocessing pipeline, generates a prediction for each valid message, and handles the message structure appropriately	15%
Static analytical report	Generates a static report from queries against the validated dimensional database, with relevant visualizations and interpreted insights.	8%
Real-time prediction dashboard	Displays predictions generated through the Kafka consumer and updates as messages arrive; it is distinct from the static report.	15%
Technical documentation	Documents both consumption paths, the Airflow DAG, data model, ML workflow, Kafka inference, setup, assumptions, and limitations; diagram matches implementation.	7%
Repository and code quality	Uploads code, previously created EDA notebooks, and diagrams to a GitHub portfolio project. Includes a properly configured .gitignore file and a comprehensive README.md explaining setup instructions and key decisions.	5%

English socialization bonus: up to +0.2 points, with the final grade capped at 5.0.